

Quantum Reservoir Computing and Extreme Learning Machines

EVA: Quantum Machine Learning – Lecture 11

Pavel Sulimov

Spring 2026

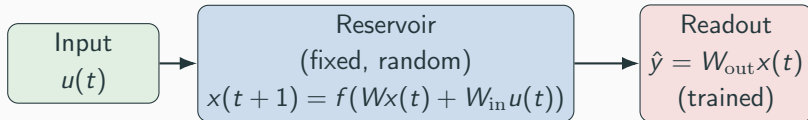
ZHAW Zürcher Hochschule für Angewandte Wissenschaften

1. Classical reservoir computing: the paradigm
2. Quantum reservoir computing (QRC): construction and readout
3. Noise as a computational resource
4. Quantum Extreme Learning Machines (QELM)
5. Time series and financial applications
6. The quantum reservoir kernel (QuaRK)

Part 1. Classical reservoir computing

The reservoir computing paradigm

Reservoir computing (RC) is a framework for temporal processing with a fixed nonlinear dynamical system:



Only W_{out} is trained – typically via linear regression. The reservoir is **never optimized**.

Key requirements on the reservoir:

- nonlinear dynamics (to mix and expand the input signal)
- fading memory (recent inputs matter more than distant ones)
- rich state space (high-dimensional to separate input patterns)

Echo State Networks and Liquid State Machines

Two classical instantiations of reservoir computing:

Echo State Network (ESN): The reservoir is a large (100–1000 unit) recurrent neural network with random, sparse connections. The recurrent weight matrix is scaled so that its spectral radius is just below 1 (the “edge of chaos” regime), ensuring fading memory.

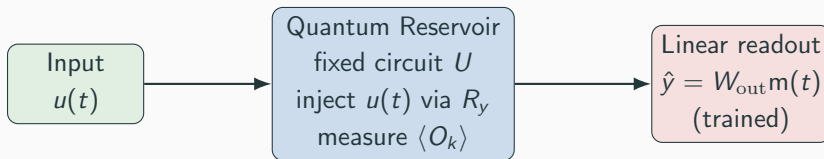
Liquid State Machine (LSM): Proposed by Maass et al. (2002); the reservoir is a spiking neural network. Input is encoded as spike trains; the output is a linear function of the population spike rate at each time step.

Both approaches train only the readout layer (linear regression or ridge regression), which is extremely fast compared to full backpropagation through time. This avoids the vanishing/exploding gradient problem for long time series.

Part 2. Quantum Reservoir Computing

QRC: replacing the reservoir with a quantum system

Fujii and Nakajima (2017) and Nakajima et al. (2019) proposed using a quantum system as the reservoir:

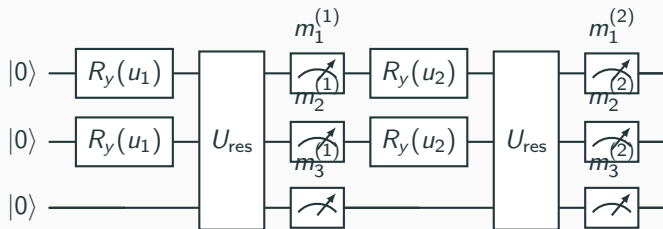


The quantum system provides:

- high-dimensional state space: 2^n complex amplitudes for n qubits
- intrinsically nonlinear dynamics (quantum evolution + measurement)
- rich correlations via entanglement – potentially harder to simulate classically

QRC: sequential input encoding

For a time series $(u_1, u_2, \dots, u_T)$, input is injected step by step:



After each input injection and fixed evolution, all qubits are measured. The measurement vector $\mathbf{m}(t) = (\langle X_1 \rangle, \langle Y_1 \rangle, \langle Z_1 \rangle, \dots, \langle Z_n \rangle)$ at time t serves as the reservoir state. The linear readout is fitted across time steps.

lecture_11_demo.ipynb, Demo 1: QRC on the Mackey-Glass time series.

QRC: fading memory

For a reservoir to be useful for time series, it must have **fading memory**: the current state must depend on recent inputs, but not on arbitrarily distant ones.

In QRC, fading memory arises naturally from measurement:

- measuring the qubits at each step partially collapses the state, erasing information about very old inputs
- re-initializing a subset of qubits after measurement limits the memory depth to a controllable window
- the memory depth is controlled by the measurement frequency and the number of qubits reset at each step

Without measurement, a unitary quantum system has no fading memory – it is reversible and perfectly recalls all past inputs. Measurement (or coupling to an environment) is therefore essential for the reservoir functionality.

Advantages of quantum reservoirs

Property	Why it helps
No barren plateaus	The reservoir is not trained; no gradient optimization over the circuit.
Exponential state space	n qubits provide a 2^n -dimensional feature space for the readout.
Quantum correlations	Entangled states produce measurement outcomes with non-classical correlations, potentially enriching the feature set.
Noise as a resource	At low noise levels, decoherence acts like stochastic regularization. See Part 3.
Fast training	Readout is linear regression – closed-form solution, no iterative optimization.

Part 3. Noise as a computational resource

Noise-induced reservoir computing

Suzuki et al. and the IBM research group showed that depolarizing and dephasing noise on quantum gates can **improve** reservoir computing performance at short circuit depths.

Intuition:

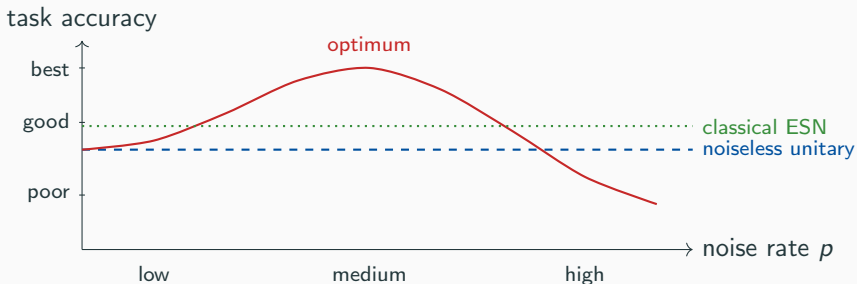
- a noiseless unitary is reversible and deterministic; adding noise introduces stochasticity that mimics a random ensemble of circuits
- stochastic averaging expands the effective feature set beyond what a single deterministic circuit provides
- this is analogous to dropout in classical NNs acting as an implicit ensemble of networks

IBM (2023) demonstrated on 156-qubit Heron processors that noise-induced QRC achieves competitive accuracy on chaotic time-series prediction (Lorenz attractor, Mackey-Glass), outperforming purely classical echo-state networks of comparable size.

Caveat: the effect is non-monotone. At high noise, the reservoir state converges to the

Optimal noise regime

Schematic of reservoir performance vs noise rate p (depolarizing):



Practical implication: calibrating the noise level (gate error rate, qubit connectivity, circuit depth) is as important as choosing the circuit structure.

Part 4. Quantum Extreme Learning Machines

QELM: the simplest QRC variant

A Quantum Extreme Learning Machine (QELM) is a single-shot (no temporal feedback) version of QRC:

1. Encode the entire input vector $x \in \mathbb{R}^d$ into a quantum state via angle encoding on n qubits.
2. Apply a fixed random circuit U .
3. Measure a set of observables $O_1, \dots, O_m$ (e.g. all single-qubit and pairwise Pauli expectations).
4. Use $(\langle O_1 \rangle, \dots, \langle O_m \rangle)$ as features for a classical linear model (ridge regression or logistic regression).

No circuit training required. The only optimization is fitting the linear readout. Training complexity: $\mathcal{O}(N \cdot m^2)$ for N samples and m features.

lecture_11_demo.ipynb, Demo 2: QELM on the Iris dataset.

QELM: feature dimension

For n qubits, how many Pauli observables are available?

Observable type	Example	Count
Weight-1 Paulis	X_i, Y_i, Z_i	$3n$
Weight-2 Paulis	$X_i X_j, X_i Y_j, \dots$	$15 \binom{n}{2}$
All Paulis up to weight 2	–	$3n + 15 \binom{n}{2}$
All non-identity Paulis	–	$4^n - 1$

For $n = 4$ qubits: $3 \cdot 4 + 15 \cdot 6 = 102$ features from weight-2 Paulis alone. Using all $4^4 - 1 = 255$ Paulis gives a 255-dimensional feature vector from a 4-qubit circuit.

Practical rule: weight-1 and weight-2 Paulis are sufficient for most tasks. Higher-weight Paulis require exponentially more circuit evaluations with shot-based simulators or hardware.

Why quantum circuits are good feature maps

A random quantum circuit of depth L on n qubits generates a feature map $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ (via Pauli expectations) with several useful properties:

- **Highly non-linear:** the map from input angles to Pauli expectations involves products of trigonometric functions at multiple frequencies.
- **Approximately orthogonal features:** random circuits produce measurements that are approximately uncorrelated for random inputs (“typicality” of quantum states).
- **Exponential in n :** the full set of Pauli expectations spans a 4^n -dimensional space; classically simulating this is hard for large n .
- **Easy to invert:** the linear readout is a closed-form problem, unlike backpropagation through the circuit.

Note: the “hardness to simulate” argument is about the feature map’s expressiveness, not a guarantee of better performance on any specific task.

Part 5. Time series and financial applications

QRC for chaotic time series prediction

Standard benchmark: predict the next value of the Mackey-Glass system, a deterministic but chaotic delay-differential equation:

$$\frac{du}{dt} = \frac{\beta u(t - \tau)}{1 + u(t - \tau)^n} - \gamma u(t).$$

QRC procedure:

1. Inject past values $u(t - k), \dots, u(t)$ sequentially into the reservoir.
2. Collect Pauli expectations $m(t)$ after each injection.
3. Fit $\hat{u}(t + 1) = w^\top m(t)$ by ridge regression on a training window.
4. Evaluate on a held-out test window.

Competitive results were obtained with as few as 4–6 qubits on a simulator, and with 8–16 qubits on IBM superconducting hardware (with error mitigation).

Financial applications: realized volatility

A 2024 study applied QRC to realized volatility forecasting on equity indices.

Setup:

- target: next-day realized volatility σ_t^2 estimated from intra-day returns
- features: QRC readout from a 4-qubit reservoir encoding 5-day lagged log-volatility values
- baseline: HAR-RV model (standard econometric benchmark)

Results:

- QRC matched HAR-RV accuracy on the SPX index
- QRC slightly outperformed on assets with fat-tailed return distributions
- the QRC readout is interpretable: large weights on weight-2 correlators suggest the circuit exploits cross-qubit correlations not captured by AR terms

Honest caveat: single-study results on financial data should be interpreted with caution;

QRC scaling on IBM hardware

A 2024–2025 IBM study evaluated QRC scaling on Heron r2 processors (up to 156 qubits):

Qubit count	Hilbert space dim	Pauli features	NMSE (Lorenz)
8	256	48	0.041
16	65 536	144	0.018
32	4×10^9	528	0.011
64	10^{19}	2016	0.009

Normalized mean squared error (NMSE) decreases as qubit count grows. Beyond 32 qubits, classical simulation of the reservoir is intractable, suggesting this regime is genuinely quantum-native computation.

Noise mitigation (Pauli twirling + ZNE) was necessary to maintain reservoir fidelity at qubit counts above 32.

Part 6. The quantum reservoir kernel (QuaRK)

QuaRK: connecting QRC and kernel methods

The quantum reservoir feature map $\phi(x) = (\langle O_1 \rangle_x, \dots, \langle O_m \rangle_x)$ induces a kernel:

$$K_{\text{QuaRK}}(x, x') = \phi(x)^\top \phi(x') = \sum_{k=1}^m \langle O_k \rangle_x \langle O_k \rangle_{x'}.$$

Properties:

- this is a Mercer kernel (positive semi-definite) by construction
- it is not the quantum fidelity kernel $|\langle \psi(x) | \psi(x') \rangle|^2$ – it uses expectation values, not state overlaps
- the kernel depends on which observables are included in ϕ ; using all Pauli strings up to weight w defines a truncated kernel
- the kernel can be computed without a quantum computer by simulating the circuit for small n (but scales exponentially with n)

QuaRK vs fidelity kernel

Property	Fidelity kernel $ \langle \psi(x) \psi(x') \rangle ^2$	QuaRK kernel $\phi(x)^\top \phi(x')$
Measurement cost	$\mathcal{O}(N^2)$ circuit evaluations	$\mathcal{O}(N \cdot m)$
Features used	State overlap	Pauli expectations
Expressiveness	Full Hilbert space	Truncated Pauli space
Interpretability	Low	High (Pauli weights)
Concentration issue	Can concentrate (plateau)	Partial: weight-1 features are safe

QuaRK is computationally cheaper for large N because the feature map is computed once per sample, not once per sample pair. The kernel matrix evaluation cost is then $\mathcal{O}(N^2 m)$ by inner product, dominated by the classical part.

Summary

Topic	Key takeaway
Classical RC	Fixed random reservoir + trained linear readout
QRC	Quantum system replaces reservoir; fading memory via measurement
Noise as resource	Optimal noise level boosts performance; high noise destroys it
IBM 156-qubit QRC	Largest QRC experiment; NMSE improves with qubit count
QELM	Single-shot encoding + fixed circuit + linear regression; no gradient
Pauli feature space	$3n + 15\binom{n}{2}$ weight-1/2 features; practical and interpretable
Financial apps	Competitive with HAR-RV on volatility forecasting; early stage
QuaRK	Reservoir feature inner product = kernel; cheaper than fidelity kernel

Lecture 12 turns to **generative models, QAOA, and QPINNs**: quantum circuits as optimization and physics-informed function approximators.